Assignment Title: Mini project [Query with SQLite, MapReduce, parallel programming]
Student Name: Maddison Turner
Student ID: 23064847
Module Name and Code: 5PAM2029-0901-2025 - Big Data and High-Performance Computing

Part 1: Creating a Database and Querying the Data

Task 1: Database Manipulation and Query

Description: In this task I used the SQLite environment on the UHHPC to query and manipulate data about different radio galaxies. The task involved creating a subset of data that matched the following restrictions: total flux greater than one, source size greater than five and separation is not null. With this new subset I created a new csv file and copied it from the UHHPC to my machine allowing me to import it into Python for further tasks.

Code:

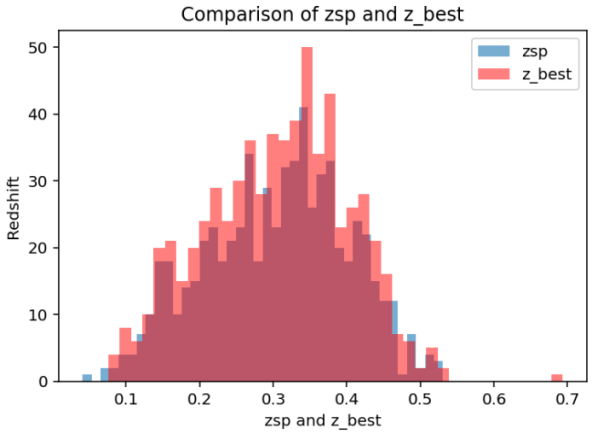
```
Sqlite3 #opening environment
.mode csv #for importing a csv file
.import LoTSS-DR2-cat_excHETDEX.csv galaxies #importing csv file as galaxies
CREATE TABLE sub_gal AS SELECT * FROM galaxies WHERE Total_flux > 1 AND Source_size > 5 AND Separation IS NOT NULL;
#creating a new table named sub_gal which contains the galaxies that total flux >1, source size >5, and separation is not null.
#exporting the new galaxy subset as 'sub_gal.csv'
.headers ON
.output sub_gal.csv
SELECT * FROM sub_gal;
.output stdout
.quit #leaving the environment

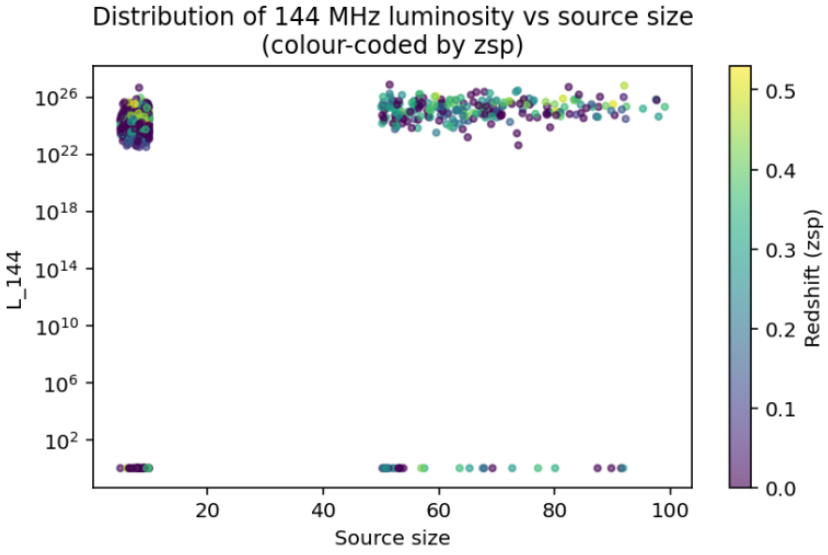
scp mt24abs@uhhpc.herts.ac.uk:/home2/mt24abs/sqlite_ass/sub_gal.csv .
#copies the file sub_gal.csv from the headnode to my machine.
```

Task 2: Exploratory Data Analysis (EDA)

Description: For this task I had to produce two plots from the sampled data that was previously queried. For the first plot I used Python libraries pandas and matplotlib to produce a histogram comparing the zsp and the z_best data. For the second plot I visualized the distribution of the data L_144 as a function of Source_size, which was colour coded by zsp data.

Plots:





Part 2: MapReduce with Hadoop

Task 1: Document Cleaning

Description: removing unnecessary words and punctuation from the thesis file. I then used Hadoop map reduce on google colab to give me an output file of the total word count for all the different words. From the output file I used python to create a table of the top and bottom ten words that where used. As well as creating a word cloud to visually show how often each word was used.

Code:

```
#importing the necessary libraries
import re
from collections import Counter
```

```
data = 'DOHERTY_PHD_thesis.txt'
with open(data, 'r', encoding = 'utf-8', errors = 'ignore') as file:
#reading the file
  text = file.read().lower()#makes everything lowercase
text = re.sub(r'[^a-z\s]', '', text)
#keeping only the letters a-z, no numbers
#everything else is gonna be replaced with a space
removed = {"section", "sections", "table", "tables", "figure", "figures", "plot", "plots",
  "panel", "panels", "shows", "show", "present", "presents", "presented",
  "demonstrates", "demonstrate", "demonstrated", "chapter", "chapters",
  "introduction", "conclusion", "conclusions", "discussion", "abstract",
  "acknowledgements", "contents", "list", "references", "the", "and", "for",
  "with", "that", "this", "from", "are", "was", "were", "can",
  "has", "have", "had", "but", "not", "which", "also", "their", "its", "than",
  "such", "these", "those", "using", "used", "use", "into", "over", "between"}
#removing simple everyday words
words = [w for w in text.split() if len(w) > 2 and w not in removed]
counter = Counter(words)#counting frequency for each word
with open('cleaned_thesis.txt', "w", encoding="utf-8") as file:
  file.write(" ".join(words))
#saving the ouput of program as a txt file that can be used for the word cloud
```

Task 2: MapReduce Program

MapReduce run:

```
!hadoop jar /content/hadoop-3.3.6/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
-files /content/drive/MyDrive/big_data/cw/mapper.py,/content/drive/MyDrive/big_data/cw/reducer.py
-input /content/drive/MyDrive/big_data/cw/cleaned_thesis.txt \
 -output /content/drive/MyDrive/big_data/cw/wordcount_output \
 -mapper "python3 mapper.py" \
 -reducer "python3 reducer.py"
```

Mapper Code:

```
#importing necessary files
import sys
import re
for line in sys.stdin:#reading each line from standard input(stdin)
  line = line.lower()#making all words lowercase
  line = re.sub(r'[^a-z\s]', '', line)#removing punctuation/numbers/uppercase
  for word in line.split():#splitting the line into each word
    print(f'{word}\t1')#gives each word a count of one
    #this is then passed onto the reducer where it totals the words
```

Reducer Code:

```
#importing the necessary library
import sys
current_word = None#initial word is set as none (0)
current_count = 0#initial word count is set to zero
for line in sys.stdin:#reading the input as standard input (stdin)
  word, count = line.strip().split('\t')
  count = int(count)#making count an integer
  if word == current_word:#if the word is the same as the last one
    current_count += count#the word count is increased by one
  else:#otherwise the word count is reset for the new word
    if current_word:#and the lasts word and its word count is printed
      print(f'{current_word}\t{current_count}')
    current_word = word
    current_count = count
if current_word:
  print(f'{current_word}\t{current_count}')
```

Task 3: Steps, Challenges, and Lessons Learned

Steps Taken: I first had to: mount my google drive, install Java, install Hadoop, extract Hadoop, and import os before I could begin my MapReduce code. I then made a new directory for all the output files to be saved too. I also had to upload the necessary files to my google drive. For example, the mapper and reducer python files.

Challenges and Lessons Learned: I kept making small mistakes like forgetting a comer or not spelling a word correctly which caused the code to not run. This has made me more observant and I quicker at debugging code. I also struggled with creating the word cloud as my python programme didn't have the correct libraries installed. To get around this I used an online editor called Replit to form the word cloud and save it to my directory. From this task I have found that using running a MapReduce on Hadoop is not that hard and if I was to complete the task again in the future, I believe that I would be a lot faster and knowledgeable about it.

Task 4: Results and Visualisation

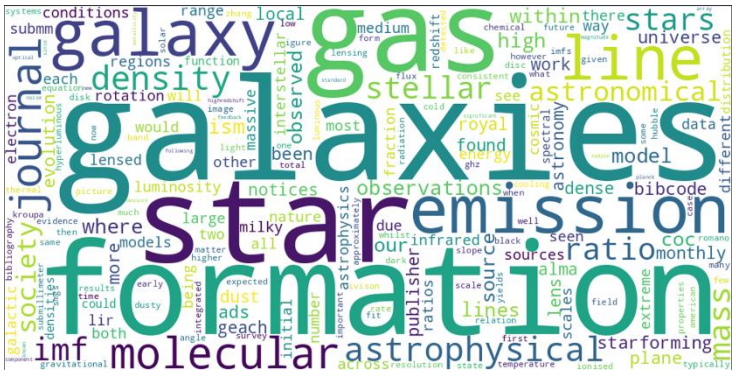
Top 10 Words Table and word cloud:

Bottom 10 words:

	Word	Count
3762	yield	1
17	ability	1
16	abia	1
15	abhkps	1
12	abel	1
6	aalp	1
5	aaf	1
4	aaap	1
3	aaam	1
2	aaak	1

Top words:

	Word	Count
1366	galaxies	237
1300	formation	189
3220	star	183
1378	gas	165
1062	emission	142
1367	galaxy	127
1936	line	115
2179	molecular	107
1805	journal	103
265	astrophysical	102



Part 3: Simple Parallel Programming and Submission to UHHPC

Task 1: Data Preparation and the Model

Description:

For this task I was asked to write a Python programme that loaded the dataset about star-forming galaxies so a linear function could be fitted. I first needed to copy the data from the UHHPC to my machine which I did by using the 'scp' command. I then used multiple different libraries to clean, model, and visualize data. For example, to import the data into Python and removing the NaN values I used the pandas library; to visualize the data into a scatter graph I used matplotlib library. For the linear function I used log base ten on the y-function (L144) and found the error propagation using the equation: $\log_{10}(y_err) = y_err / (y * \log_{10})$. By doing this I could form a linear function which served as a line of best fit for the scatter graph plot.

Code:

```
#importing the libraries needed
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit
stars = pd.read_csv('stars.csv')#reading and selecting the data needed
stars_clean = stars.dropna(subset = ['SFR', 'L144', 'E_L144'])#removing the NaN values
stars_clean = stars[stars['L144']>0]#removes bad logs
x = stars_clean['SFR']#x = star formation rate
y = stars_clean['L144']#y = 144 MHz luminosity
yerr = stars_clean['E_L144']#y_err = error on 144 MHz luminosity
logy = np.log10(y)#logging luminosity to base 10
#error propagation:σ_z = σ_y / ylog10, σ_z = log10y_err, σ_y = y_err
log10y_err = yerr / y * np.log(10)
#linear model will be logy =  mx + c
def linear_model(x, m, c):
  return m * x + c

#fitting the linear model
popt, pcov = curve_fit(linear_model, x, logy, sigma = log10y_err, absolute_sigma = True)
m , c = popt
m_err, c_err = np.sqrt(np.diag(pcov))
n = len(x)#number of data points
#creating the line of best fit
```

```
xfit = np.linspace(min(x), max(x), n)
yfit = linear_model(xfit, m, c)
print('Results:')
print(f'Slope = {m:.3f} ± {m_err:.3f}')
print(f'Intercept = {c:.3f} ± {c_err:.3f}')
print(f'Number of data points = {n}')
```

Task 2: Parallel Programming Implementation

Description:

I have used multiprocessing for my parallelisation method because I am completing multiple independent linear fits. Although it is CPU intensive, since it automatically uses all the available cores, it means that each task has its own Python interpreter and runs on its own CPU core. I also decided to use multiprocessing over MPI (mpi4py) since it is less time consuming and easier to setup. I played around with the different number of bootstrap resamples (can be seen below) to see how each would affect the speed and overall outcome. I made the decision that the number 1000 was the best one to go with as it provides a better outcome of uncertainty for the slope and intercept whilst not being too time intensive.

Bootstrap resamples	Slope	Intercept	Runtime (seconds)
500	0.824 ± 0.184	22.494 ± 0.132	0.21
1000	0.819 ± 0.184	22.497 ± 0.131	0.27
2000	0.820 ± 0.184	22.495 ± 0.133	0.47

Code:

```
#importing the necessary libraries
from multiprocessing import Pool, cpu_count
import time
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit
stars = pd.read_csv('stars.csv')#reading and selecting the data needed
stars_clean = stars.dropna(subset = ['SFR', 'L144', 'E_L144'])#removing the NaN values
stars_clean = stars_clean[stars_clean['L144']>0]#removes bad logs
x = stars_clean['SFR'].values#x = star formation rate
y = stars_clean['L144'].values#y = 144 MHz luminosity
yerr = stars_clean['E_L144'].values#y_err = error on 144 MHz luminosity
n = len(x)#number of data points
logy = np.log10(y)#logging luminosity to base 10
#error propagation:
# σ_z = σ_y / ylog10
#σ_z = log10y_err, σ_y = y_err
log10y_err = yerr / (y * np.log(10))
#linear model will be logy =  mx + c
def linear_model(x, m, c):
  return m * x + c
def fit_once(seed):
  np.random.seed(seed)
  idx = np.random.randint(0, n, n)
  popt, _ = curve_fit(
    linear_model,
    x[idx],
    logy[idx],
    sigma=log10y_err[idx],
    absolute_sigma=True)
  return popt#returns the fitted slope and intercept
if __name__ == '__main__':#required for multiprocessing
  start = time.time()#start time
  n_bootstrap = 1000#number of bootstrap fits to perform
  n_cores = cpu_count()
  with Pool(processes = n_cores) as pool:#creates a pool of workers
```

```
    #divides bootstrap fits across the cpu cores
    results = pool.map(fit_once, range(n_bootstrap))
  results = np.array(results)#makes the results an array
  #seperate the slope and intercept values
  mvals = results[:, 0]
  cvals = results[:, 1]
  end = time.time()#ending time
  running = end - start#finding the run time
  #printing the results to 3d.p
  print('Parallel Linear Fit Results')
  print('--------------------------')
  print(f'Number of bootstrap fits: {n_bootstrap}')
  print(f'Number of cores: {n_cores}')
  print(f'Slope (m): {np.mean(mvals):.3f} ± {np.std(mvals):.3f}')
  print(f'Intercept (c): {np.mean(cvals):.3f} ± {np.std(cvals):.3f}')
  print(f'Total runtime: {running:.2f} seconds')
#job submission file
#!/bin/bash
#SBATCH --job-name=parallel_job
#SBATCH --output=parallel_output.txt
#SBATCH --error=stars_error.txt
#SBATCH --time=01:00:00
#SBATCH --partition=core32
#SBATCH --nodes=1
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G
#SBATCH --mail-user=mt24abs@herts.ac.uk
#SBATCH --mail-type=ALL
python3 parallel.py
sbatch parallel.sh
```

Task 3: Results and Visualisation

Results: linear results and parallel processing results

Linear	**Symbol**	**Value**
Slope	m	0.824 ± 0.004
Intercept	c	22.513 ± 0.003
Number of data points	n	3400

Parallel processing	**Symbol**	**Value**
Slope	m	0.819 ± 0.184
Intercept	c	22.497 ± 0.131
Number of data points	n	3400
Processing time		0.27 seconds

Scatter Plot:

